

目次

一、RESTful	1
REST	1
Resource：資源	1
Representation：表現形式	2
State Transfer：狀態轉移	3
REST 的主要架構限制	3
Client-Server	3
Stateless	4
Cacheable	4
Uniform Interface	5
Layered System	5
RESTful	6
二、RESTful API	7
API	7
RESTful API 的基本組成	7
Endpoint	7
HTTP Methods 與 CRUD	8
HTTP Status Code	10
Request Body 與 Response Body	10
三、Ajax 與 JavaScript Fetch	12
Ajax	12
Fetch API	13
四、SQLite	16
Python sqlite3 套件	16
SQLite 的常見資料型態	17
Titanic 資料表欄位說明	17
建立 titanic 資料表 Schema	18
SQLite CRUD 語法	18
Create：新增資料	18
Read：查詢資料	19
Update：修改資料	19
Delete：刪除資料	20
五、建立 Flask + SQLite + Fetch CRUD 專案	21
資料庫初始化	21
Web API 程式	23
首頁	32
新增頁面	38

編輯頁面.....	41
-----------	----

課程範例網址：

https://github.com/telunyang/restful_api_ajax

一、RESTful

REST

REST 是 **Representational State Transfer** 的縮寫，中文通常可翻譯為「表現層狀態轉移」或「表述性狀態轉移」。REST 不是一套程式語言，也不是一個框架，而是一種用來設計網路系統的 **架構風格**。

在 Web 開發中，我們常常會讓前端網頁、手機 App、後端伺服器、資料庫彼此溝通。REST 的核心目的，就是讓這些系統之間可以透過一致、清楚、可預測的方式交換資料，簡單來說：REST 是一種設計網路服務的風格，它把系統中的資料視為「資源」，並透過統一的介面操作這些資源。

例如 Titanic 乘客資料可以被視為一種資源：

- /api/passengers
- /api/passengers/1
- /api/passengers/2

其中：

- /api/passengers
代表「所有乘客資料」這個資源集合。
- /api/passengers/1
代表「PassengerId 為 1 的乘客」這一筆單一資源。

Resource：資源

REST 的第一個重要概念是 **Resource**，也就是「資源」。在 REST 的觀念中，系統中的重要資料都可以被視為資源。例如：

系統情境	Resource 範例
使用者系統	users
商品系統	products
訂單系統	orders
學生系統	students
Titanic 資料集	passengers

表：系統情境與對應的 Resource 範例

REST API 通常會用 URI 表示資源，例如：

- GET /api/passengers
- GET /api/passengers/10
- POST /api/passengers
- PUT /api/passengers/10
- DELETE /api/passengers/10

在這裡，`passengers` 是資源名稱，`10` 是特定乘客的 ID。

註：可參考 [Café Nomad](#) 的 API 格式。

Representation：表現形式

REST 裡面的 Representation 指的是「資源的表現形式」，同一個資源可以用不同格式表示，例如：

```
{
  "PassengerId": 1,
  "Name": "Braund, Mr. Owen Harris",
  "Sex": "male",
  "Age": 22,
  "Survived": 0
}
```

這是 JSON 格式的乘客資料。

同一筆資料也可以被表示成：

```
<tr>
  <td>1</td>
  <td>Braund, Mr. Owen Harris</td>
  <td>male</td>
  <td>22</td>
</tr>
```

這是 HTML 表格格式的乘客資料。

REST API 最常使用 `JSON`，因為 JSON 很容易被 JavaScript、Python、Java、C#、PHP 等語言解析。

State Transfer：狀態轉移

REST 的「狀態轉移」可以理解為：Client 透過 HTTP request 向 Server 要求資源，Server 回傳資源的 representation，Client 根據回傳結果更新自己的畫面狀態。

情境	狀態轉移過程
使用者打開首頁	瀏覽器 → GET /api/passengers → Flask Server → SQLite → 回傳 JSON → JavaScript 更新表格
使用者新增資料	瀏覽器表單 → POST /api/passengers → Flask Server → SQLite INSERT → 回傳新增成功 → JavaScript 更新畫面
使用者刪除資料	瀏覽器按下刪除 → DELETE /api/passengers/5 → Flask Server → SQLite DELETE → 回傳刪除成功 → JavaScript 重新載入清單

表：狀態轉移情境

REST 的主要架構限制

Client-Server

Client 和 Server 之間彼此分工清楚，例如：

- Client 負責
 - 畫面呈現
 - 使用者互動
 - 呼叫 API
 - 處理回傳資料
- Server 負責：
 - 接收請求
 - 驗證資料
 - 執行商業邏輯
 - 存取資料庫
 - 回傳結果

角色	技術
----	----

Client	HTML + JavaScript + fetch
Server	Flask
Database	SQLite

表：Client 與 Server 之間的分工

Stateless

Stateless 是 REST 很重要的原則，意思是：每一次 request 都必須包含 Server 處理這次請求所需要的資訊，Server 不應該依賴前一次 request 的暫存狀態。例如：

- GET /api/passengers?page=2&per_page=20

這個 request 已經明確告訴 Server：

- 我要第 2 頁
- 每頁 20 筆

Server 不需要記得使用者上一頁看了什麼。

Stateless 的好處是：

- Server 比較容易擴充
- API 行為比較可預測
- 減少伺服器端狀態管理負擔
- 比較適合負載平衡與分散式系統

Cacheable

如果某些資料短時間內不會改變，Server 可以告訴 Client 或瀏覽器是否可以快取資料，例如：

- GET /api/passengers/1

這種查詢單筆乘客資料的 API，在資料不常變動的情況下，可以被快取，但如果是新增、修改、刪除資料的 API，例如：

- POST /api/passengers
- PUT /api/passengers/1
- DELETE /api/passengers/1

通常不適合直接快取。

Uniform Interface

Uniform Interface 是 REST 的核心概念，意思是系統應該用一致的方式操作資源，例如：

操作	HTTP Method	URI
取得所有乘客	GET	/api/passengers
取得單一乘客	GET	/api/passengers/1
新增乘客	POST	/api/passengers
修改乘客	PUT	/api/passengers/1
刪除乘客	DELETE	/api/passengers/1

表：使用一致的方法來操作資源

這種設計比下面這種方式更 RESTful：

- /getAllPassengers
- /getPassengerById
- /createPassenger
- /updatePassenger
- /deletePassenger

RESTful API 通常不把動作寫在 URL，而是讓 HTTP method 表示動作，讓 URL 表示資源。

Layered System

REST 也允許系統被分成多層，例如：

- Browser
 - 瀏覽器或是 APP
- Frontend Server
 - CDN 或是 Proxy Server
- API Gateway
 - 不同資源的呈現方式
- Flask API Server
 - 透過 route 取得不同的資源
- Database
 - 可以是 Relational Database，或是 Not Only SQL (NoSQL)

Client 不一定知道後面有幾層，它只需要知道 API endpoint 即可。

RESTful

REST 是架構風格，而 RESTful 是形容詞。如果一個 API 的設計符合 REST 的主要原則，我們就說它是 RESTful API，例如下面這種設計比較

RESTful：

- GET /api/passengers
- GET /api/passengers/1
- POST /api/passengers
- PUT /api/passengers/1
- DELETE /api/passengers/1

下面這種設計比較像 RPC (Remote Procedure Call) 風格，而不是

RESTful：

- /api/getPassenger
- /api/createPassenger
- /api/updatePassenger
- /api/deletePassenger

RESTful API 的重點是：

- URL 表示資源
- HTTP method 表示操作
- JSON 表示資料
- HTTP status code 表示結果
- 每次 request 都要能獨立處理

二、RESTful API

API

API 是 Application Programming Interface，中文可以翻譯為「應用程式介面」，API 的意思是：一個系統提供給另一個系統使用的操作介面。Flask Server 會提供 API 給前端 JavaScript 使用。前端不需要知道 SQLite 怎麼查詢，也不需要知道後端 Python 程式怎麼寫，它只需要呼叫 API，例如：

```
fetch("/api/passengers")
```

這一行會向後端取得乘客資料。

RESTful API 的基本組成

一個 RESTful API 通常包含以下元素：

- Endpoint
- HTTP Method
- Request
- Response
- Status Code
- Header
- Body

Endpoint

- 意思：它代表你要求或操作的「資源」在哪裡。
- 餐廳比喻：就像菜單上的「各個菜單分類」（如：飲料區、主餐區）。
- 範例：`https://api.restaurant.com/v1/dishes`（這代表「餐點」這個資源的端點）

Endpoint 是 API 的網址路徑，例如：

- `/api/passengers`
- `/api/passengers/1`

Endpoint 通常應該使用名詞，而不是動詞。

建議	不建議
/api/passengers	/api/getPassengers
/api/products	/api/createPassenger
/api/orders	/api/deletePassenger

表：Endpoint 的設計應符合建議的格式

HTTP Methods 與 CRUD

CRUD 是資料庫常見的四種操作：

CRUD	意思	SQL	HTTP Method
Create	新增	INSERT	POST
Read	讀取	SELECT	GET
Update	修改	UPDATE	PUT/PATCH
Delete	刪除	DELETE	DELETE

以 Titanic passengers API 為例：

功能	Method	Endpoint	說明
查詢乘客清單	GET	/api/passengers	取得多筆資料
查詢單一乘客	GET	/api/passengers/1	取得一筆資料
新增乘客	POST	/api/passengers	新增一筆資料
完整修改乘客	PUT	/api/passengers/1	更新一整筆資料
部分修改乘客	PATCH	/api/passengers/1	更新部分欄位
刪除乘客	DELETE	/api/passengers/1	刪除一筆資料

GET 用來讀取資料，例如：

- GET /api/passengers

回傳：

```
{
  "items": [
    {
      "PassengerId": 1,
      "Name": "Braund, Mr. Owen Harris",
      "Sex": "male",
      "Age": 22,
```

```

        "Survived": 0
    }
],
"page": 1,
"per_page": 20,
"total": 891
}

```

注意：GET 不應該修改資料庫內容。

POST 用來新增資料，例如：

- POST /api/passengers
 - Content-Type: application/json

Request body：

```

{
    "Survived": 1,
    "Pclass": 1,
    "Name": "New Passenger",
    "Sex": "female",
    "Age": 30,
    "SibSp": 0,
    "Parch": 0,
    "Ticket": "NEW123",
    "Fare": 100,
    "Cabin": "C100",
    "Embarked": "S"
}

```

Server 新增成功後，可以回傳：

```

{
    "message": "created",
    "item": {
        "PassengerId": 892,
        "Name": "New Passenger"
    }
}

```

PUT 通常用來完整更新一個資源。

- PUT /api/passengers/1

PATCH 通常用來部分更新一個資源。

- PATCH /api/passengers/1

DELETE 用來刪除資料，例如：

- DELETE /api/passengers/1

刪除成功後，可以回傳：

```
{  
  "message": "deleted"  
}
```

HTTP Status Code

RESTful API 應該使用 HTTP status code 表示處理結果。

Status Code	意思	使用情境
200	OK	查詢成功、修改成功
201	Created	新增成功
204	No Content	刪除成功但不回傳內容
400	Bad Request	前端送來的資料格式錯誤
401	Unauthorized	請求尚未驗證
403	Forbidden	理解請求但拒絕執行它
404	Not Found	找不到指定資源
405	Method Not Allowed	請求方法不能用於對應資源
409	Conflict	資料衝突，例如重複 ID
500	Internal Server Error	後端程式錯誤

表：常用狀態碼，取自 <https://zh.wikipedia.org/zh-tw/HTTP%E7%8A%B6%E6%80%81%E7%A0%81>

Request Body 與 Response Body

前端送資料給後端時，常用 JSON 格式。

```
fetch("/api/passengers", {
```

```
method: "POST",
headers: {
    "Content-Type": "application/json"
},
body: JSON.stringify(data)
});
```

後端 Flask 可以用：

```
from flask import request
data = request.get_json()
```

取得 JSON body。

Flask 回傳 JSON 時可以使用：

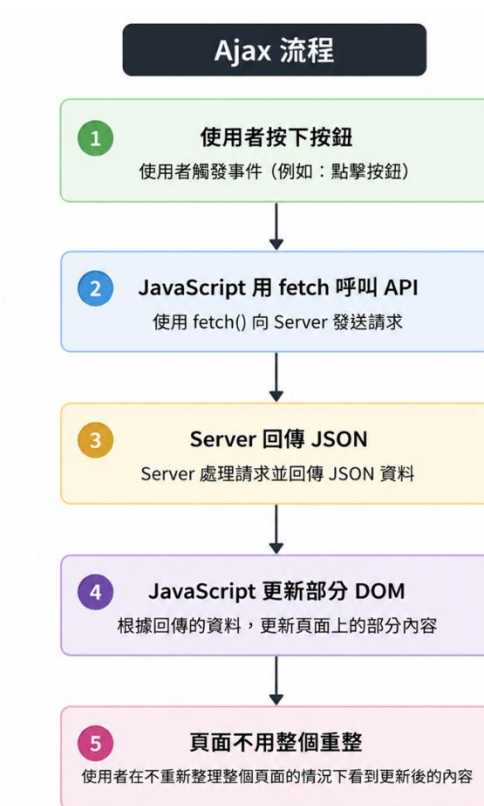
```
from flask import jsonify
return jsonify({"message": "created"}), 201
```

或直接回傳 dictionary。

三、Ajax 與 JavaScript Fetch

Ajax

Ajax 是 Asynchronous JavaScript and XML 的縮寫。雖然名稱裡面有 XML，但現代 Ajax 開發大多使用 JSON，而不是 XML。其核心概念是：網頁可以在不重新整理整個頁面的情況下，向 Server 取得資料或送出資料。



圖：Ajax 流程

Ajax 的優點包括：

- 使用者體驗較好
- 不需要每次操作都重新載入整個頁面
- 前後端分工清楚
- 適合串接 RESTful API
- 可以動態更新表格、表單、圖表與清單

Fetch API

Fetch API 是瀏覽器提供的 JavaScript API，用來送出 HTTP request。

基本語法：

```
<script>
fetch("/api/passengers")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
</script>
```

現代 JavaScript 更常使用 async/await：

```
async function loadPassengers() {
  const response = await fetch("/api/passengers");
  const data = await response.json();
  console.log(data);
}
```

Fetch 的 GET 範例

```
async function loadPassengers() {
  const response = await fetch("/api/passengers");

  if (!response.ok) {
    alert("資料讀取失敗");
    return;
  }

  const data = await response.json();
  console.log(data.items);
}
```

重點：

- fetch() 會送出 HTTP request
- await 會等待 response 回來
- response.ok 可以判斷 HTTP status 是否成功
- response.json() 可以把 JSON 字串轉成 JavaScript object

Fetch 的 POST 範例

```
async function createPassenger(data) {
  const response = await fetch("/api/passengers", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(data)
  });

  const result = await response.json();

  if (!response.ok) {
    alert(result.error || "新增失敗");
    return;
  }

  alert("新增成功");
}
```

Fetch 的 PUT 範例

```
async function updatePassenger(passengerId, data) {
  const response = await fetch(`/api/passengers/${passengerId}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(data)
  });

  const result = await response.json();

  if (!response.ok) {
    alert(result.error || "更新失敗");
    return;
  }
}
```



```
    alert("更新成功");  
}
```

Fetch 的 DELETE 範例

```
async function deletePassenger(passengerId) {  
    const response = await fetch(`/api/passengers/${passengerId}`, {  
        method: "DELETE"  
    });  
  
    if (!response.ok) {  
        alert("刪除失敗");  
        return;  
    }  
  
    alert("刪除成功");  
}
```

四、SQLite

SQLite 是一種輕量級關聯式資料庫。它不需要另外安裝資料庫伺服器，資料會直接儲存在一個 .db 檔案中。

SQLite 適合用在：

- 教學
- 小型網站
- 桌面應用程式
- 單機資料儲存
- 原型系統開發
- 測試環境

Python sqlite3 套件

Python 內建 sqlite3 套件，因此通常不需要額外安裝。基本流程如下：

```
import sqlite3

conn = sqlite3.connect("my_db.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM titanic")

rows = cursor.fetchall()

conn.close()
```

常見操作流程：

- connect：連線到資料庫
- cursor：建立 SQL 執行物件
- execute：執行 SQL
- fetchone / fetchall：取得查詢結果
- commit：確認新增、修改、刪除
- close：關閉連線

SQLite 的常見資料型態

常見 storage class 包含：

型態	說明	範例
INTEGER	整數	1, 2, 100
REAL	浮點數	22.5, 71.2833
TEXT	文字	"male", "female"
BLOB	二進位資料	圖片、檔案
NULL	空值	NULL

SQLite 的型態系統比較彈性，建議明確設計欄位型態與限制。

Titanic 資料表欄位說明

欄位名稱	建議型態	是否 NULL	值域或長度	說明
PassengerId	INTEGER	否	1-891	乘客 ID，可作為主鍵
Survived	INTEGER	否	0 或 1	是否生還，0=未生還，1=生還
Pclass	INTEGER	否	1, 2, 3	艙等
Name	TEXT	否	原始最大長度約 82，可設 100	乘客姓名
Sex	TEXT	否	male / female	性別
Age	REAL	是	約 0.42-80	年齡
SibSp	INTEGER	否	0-8	船上兄弟姊妹或配偶數
Parch	INTEGER	否	0-6	船上父母或子女數
Ticket	TEXT	否	原始最大長度約 18，可設 30	船票號碼
Fare	REAL	否	0-512.3292	票價
Cabin	TEXT	是	原始最大長度約 15，可設 30	客艙
Embarked	TEXT	是	C / Q / S	登船港口

表：資料共有 891 筆、12 個欄位。

建立 titanic 資料表 Schema

```
CREATE TABLE IF NOT EXISTS titanic (  
    PassengerId INTEGER PRIMARY KEY,  
    Survived INTEGER NOT NULL CHECK (Survived IN (0, 1)),  
    Pclass INTEGER NOT NULL CHECK (Pclass IN (1, 2, 3)),  
    Name TEXT NOT NULL CHECK (length(Name) <= 100),  
    Sex TEXT NOT NULL CHECK (Sex IN ('male', 'female')),  
    Age REAL CHECK (Age IS NULL OR (Age >= 0 AND Age <= 120)),  
    SibSp INTEGER NOT NULL DEFAULT 0 CHECK (SibSp >= 0),  
    Parch INTEGER NOT NULL DEFAULT 0 CHECK (Parch >= 0),  
    Ticket TEXT NOT NULL CHECK (length(Ticket) <= 30),  
    Fare REAL NOT NULL DEFAULT 0 CHECK (Fare >= 0),  
    Cabin TEXT CHECK (Cabin IS NULL OR length(Cabin) <= 30),  
    Embarked TEXT CHECK (Embarked IS NULL OR Embarked IN ('C', 'Q',  
'S'))  
);
```

這個 schema 使用了幾種重要限制：

SQL 限制	說明
PRIMARY KEY	主鍵，每筆資料唯一識別
NOT NULL	欄位不可為空
CHECK	檢查欄位值是否合法
DEFAULT	沒有提供值時使用預設值

SQLite CRUD 語法

Create：新增資料

```
INSERT INTO titanic (  
    Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin,  
    Embarked  
) VALUES (  
    1, 1, 'New Passenger', 'female', 30, 0, 0, 'NEW123', 100.0, 'C100',  
    'S'
```

);

Read：查詢資料

查詢全部：

```
SELECT * FROM titanic;
```

查詢單筆：

```
SELECT * FROM titanic  
WHERE PassengerId = 1;
```

查詢生還乘客：

```
SELECT * FROM titanic  
WHERE Survived = 1;
```

查詢第一艙乘客：

```
SELECT * FROM titanic  
WHERE Pclass = 1;
```

分頁查詢：

```
SELECT * FROM titanic  
ORDER BY PassengerId  
LIMIT 20 OFFSET 0;
```

Update：修改資料

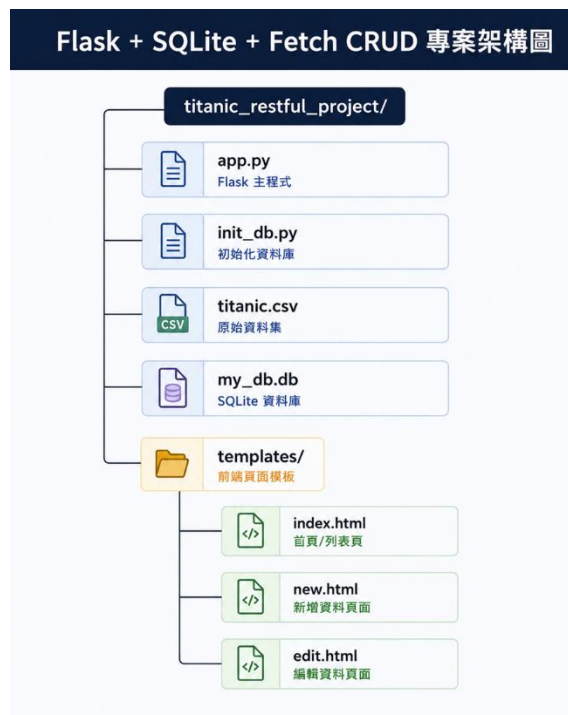
```
UPDATE titanic  
SET  
    Name = 'Updated Passenger',  
    Age = 31,  
    Fare = 120.5  
WHERE PassengerId = 1;
```

Delete：刪除資料

```
DELETE FROM titanic  
WHERE PassengerId = 1;
```

五、建立 Flask + SQLite + Fetch CRUD 專案

建立以下資料夾結構（也可以依需求而定）：



圖：專案架構圖

安裝 VS code 擴充功能：SQLite3 Editor

資料庫初始化

init_db.py

```
import sqlite3
import pandas as pd

# 資料庫檔案名稱
DB_PATH = "my_db.db"

# CSV 檔案路徑
CSV_PATH = "titanic.csv"

# 建立資料表的 SQL 語句，包含欄位定義和約束條件
```

```

CREATE_TABLE_SQL = """
CREATE TABLE IF NOT EXISTS titanic (
    PassengerId INTEGER PRIMARY KEY,
    Survived INTEGER NOT NULL CHECK (Survived IN (0, 1)),
    Pclass INTEGER NOT NULL CHECK (Pclass IN (1, 2, 3)),
    Name TEXT NOT NULL CHECK (length(Name) <= 100),
    Sex TEXT NOT NULL CHECK (Sex IN ('male', 'female')),
    Age REAL CHECK (Age IS NULL OR (Age >= 0 AND Age <= 120)),
    SibSp INTEGER NOT NULL DEFAULT 0 CHECK (SibSp >= 0),
    Parch INTEGER NOT NULL DEFAULT 0 CHECK (Parch >= 0),
    Ticket TEXT NOT NULL CHECK (length(Ticket) <= 30),
    Fare REAL NOT NULL DEFAULT 0 CHECK (Fare >= 0),
    Cabin TEXT CHECK (Cabin IS NULL OR length(Cabin) <= 30),
    Embarked TEXT CHECK (Embarked IS NULL OR Embarked IN ('C', 'Q',
'S'))
);
"""

# 建立索引的 SQL 語句，提升查詢效率
CREATE_INDEX_SQL = [
    "CREATE INDEX IF NOT EXISTS idx_titanic_survived ON
titanic(Survived);",
    "CREATE INDEX IF NOT EXISTS idx_titanic_pclass ON
titanic(Pclass);",
    "CREATE INDEX IF NOT EXISTS idx_titanic_name ON titanic(Name);"
]

# 初始化資料庫，建立資料表並匯入 CSV 資料
def init_db():
    # 連接到 SQLite 資料庫，如果資料庫不存在會自動建立
    conn = sqlite3.connect(DB_PATH)

    try:
        # 建立游標物件，用於執行 SQL 語句
        cursor = conn.cursor()

        # 如果資料表已存在，先刪除再重新建立，以確保資料表結構正確
        cursor.execute("DROP TABLE IF EXISTS titanic;")

```



```

        cursor.execute(CREATE_TABLE_SQL)

        # 建立索引，提升查詢效率
        for sql in CREATE_INDEX_SQL:
            cursor.execute(sql)

        # 使用 pandas 讀取 CSV 檔案，並將資料匯入 SQLite 資料庫
        df = pd.read_csv(CSV_PATH)

        # pandas 的 NaN 需要轉成 None，SQLite 才會存成 NULL
        df = df.where(pd.notnull(df), None)

        # 將 DataFrame 的資料匯入 SQLite 資料庫的 titanic 資料表
        df.to_sql("titanic", conn, if_exists="append", index=False)

        # 提交事務，將所有變更儲存到資料庫
        conn.commit()
    except sqlite3.Error as e:
        # 發生錯誤時回滾事務，確保資料庫不會處於不一致的狀態
        conn.rollback()
        print(f"資料庫初始化失敗: {e}")
    finally:
        # 關閉資料庫連接，釋放資源
        conn.close()
        print("my_db.db 建立完成，titanic 資料表已匯入。")

if __name__ == "__main__":
    init_db()

```

Web API 程式

```

app.py

import sqlite3
from flask import Flask, jsonify, request, render_template

app = Flask(__name__)

```

```

# =====
# 1. 全域讀取資料庫
# =====

DATABASE = "my_db.db"

# 這裡我們直接在全域讀取資料庫，這樣在每個 route 就可以直接使用 db 來存取資料庫了。
db = sqlite3.connect(DATABASE, check_same_thread=False)

# 讓我們在讀取資料庫時，可以直接用 row["欄位名稱"] 的方式來存取資料，
# 而不是 row[0]、row[1] 這樣的 index。
db.row_factory = sqlite3.Row

# =====
# 2. 小工具：把 SQLite Row 轉成 dict
# =====

def row_to_dict(row):
    return dict(row)

# =====
# 3. 前端頁面 Routes
# =====

# 首頁
@app.route("/")
def index_page():
    return render_template("index.html")

# 新增乘客頁面
@app.route("/passengers/new")
def new_passenger_page():
    return render_template("new.html")

# 編輯乘客頁面

```

```

@app.route("/passengers/<int:passenger_id>/edit")
def edit_passenger_page(passenger_id):
    return render_template("edit.html", passenger_id=passenger_id)

# =====
# 4. API：取得全部乘客資料，包含簡單分頁
# GET /api/passengers?page=1&per_page=20
# =====

@app.route("/api/passengers", methods=["GET"])
def get_passengers():
    # 讀取 query string 的 page 和 per_page 參數，並設定預設值
    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 20, type=int)

    # 搜尋姓名
    search = request.args.get("search", "")

    # 計算 SQL 查詢的 offset，用於分頁
    offset = (page - 1) * per_page

    # 根據是否有搜尋關鍵字，執行不同的 SQL 查詢
    if search != "":
        # 有輸入搜尋關鍵字：只查詢姓名符合的資料
        total_row = db.execute(
            """
            SELECT COUNT(*) AS total
            FROM titanic
            WHERE Name LIKE ?
            """,
            (f"%{search}%",)
        ).fetchone()

        rows = db.execute(
            """
            SELECT *
            FROM titanic
            WHERE Name LIKE ?

```

```

        ORDER BY PassengerId
        LIMIT ?
        OFFSET ?
        """ ,
        (f"%{search}%", per_page, offset)
    ).fetchall()

else:
    # 沒有輸入搜尋關鍵字：查詢全部資料
    total_row = db.execute(
        """
        SELECT COUNT(*) AS total
        FROM titanic
        """
    ).fetchone()

    rows = db.execute(
        """
        SELECT *
        FROM titanic
        ORDER BY PassengerId
        LIMIT ?
        OFFSET ?
        """ ,
        (per_page, offset)
    ).fetchall()

    # 從 total_row 取出 total 的值，這樣我們就知道總共有多少筆資料了。
    total = total_row["total"]

    # 最後回傳 JSON 格式的資料，包含 items（資料列表）、page、per_page 和 total。
    return jsonify({
        "items": [row_to_dict(row) for row in rows],
        "page": page,
        "per_page": per_page,
        "total": total
    })

```

```

# =====
# 5. API：取得單一乘客
# GET /api/passengers/1
# =====

@app.route("/api/passengers/<int:passenger_id>", methods=["GET"])
def get_passenger(passenger_id):
    # 根據 passenger_id 查詢資料庫，看看有沒有這個乘客的資料。
    row = db.execute(
        "SELECT * FROM titanic WHERE PassengerId = ?",
        (passenger_id,)
    ).fetchone()

    # 如果 row 是 None，代表資料庫裡沒有這個 passenger_id 的資料，我們就回傳 404 Not Found 的錯誤訊息。
    if row is None:
        return jsonify({"error": "找不到資料"}), 404

    # 如果有找到資料，我們就把這筆資料轉成 dict，然後回傳 JSON 格式的資料。
    return jsonify(row_to_dict(row))

# =====
# 6. API：新增乘客
# POST /api/passengers
# =====

@app.route("/api/passengers", methods=["POST"])
def create_passenger():
    # 從 request 的 JSON body 讀取資料
    data = request.get_json()

    # 執行 SQL INSERT 語句，把新的乘客資料新增到 titanic 資料表中。
    cursor = db.execute(
        """
        INSERT INTO titanic (
            Survived,

```

```

        Pclass,
        Name,
        Sex,
        Age,
        SibSp,
        Parch,
        Ticket,
        Fare,
        Cabin,
        Embarked
    )
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
"",
(
    data["Survived"],
    data["Pclass"],
    data["Name"],
    data["Sex"],
    data["Age"],
    data["SibSp"],
    data["Parch"],
    data["Ticket"],
    data["Fare"],
    data["Cabin"],
    data["Embarked"]
)
)

# 執行 commit(), 把剛剛的 INSERT 操作真正寫入資料庫。
db.commit()

# cursor.lastrowid 會回傳剛剛 INSERT 的那筆資料的自動增加的 ID,
# 也就是 PassengerId。
new_id = cursor.lastrowid

# 根據 new_id 查詢剛剛新增的那筆資料, 這樣我們就可以把完整的資料回傳給前端了。
row = db.execute(

```

```

        "SELECT * FROM titanic WHERE PassengerId = ?",
        (new_id,)
    ).fetchone()

    # 最後回傳 JSON 格式的資料，包含 message 和 item（剛剛新增的那筆資料）。
    return jsonify({
        "message": "created",
        "item": row_to_dict(row)
    }), 201

# =====
# 7. API：修改乘客
# PUT /api/passengers/1
# =====

@app.route("/api/passengers/<int:passenger_id>", methods=["PUT"])
def update_passenger(passenger_id):
    # 從 request 的 JSON body 讀取資料
    data = request.get_json()

    # 執行 SQL UPDATE 語句，根據 passenger_id 把對應的資料更新成新的值。
    db.execute(
        """
        UPDATE titanic
        SET
            Survived = ?,
            Pclass = ?,
            Name = ?,
            Sex = ?,
            Age = ?,
            SibSp = ?,
            Parch = ?,
            Ticket = ?,
            Fare = ?,
            Cabin = ?,
            Embarked = ?
        WHERE PassengerId = ?
    """
    )

```

```

        "",
        (
            data["Survived"],
            data["Pclass"],
            data["Name"],
            data["Sex"],
            data["Age"],
            data["SibSp"],
            data["Parch"],
            data["Ticket"],
            data["Fare"],
            data["Cabin"],
            data["Embarked"],
            passenger_id
        )
    )

    # 執行 commit(), 把剛剛的 UPDATE 操作真正寫入資料庫。
    db.commit()

    # 根據 passenger_id 查詢剛剛更新的那筆資料，這樣我們就可以把完整的資料
    回傳給前端了。
    row = db.execute(
        "SELECT * FROM titanic WHERE PassengerId = ?",
        (passenger_id,)
    ).fetchone()

    # 如果 row 是 None，代表資料庫裡沒有這個 passenger_id 的資料，我們就回
    傳 404 Not Found 的錯誤訊息。
    if row is None:
        return jsonify({"error": "找不到資料"}), 404

    # 最後回傳 JSON 格式的資料，包含 message 和 item (剛剛更新的那筆資
    料)。
    return jsonify({
        "message": "updated",
        "item": row_to_dict(row)
    })

```



```

# =====
# 8. API：刪除乘客
# DELETE /api/passengers/1
# =====

@app.route("/api/passengers/<int:passenger_id>", methods=["DELETE"])
def delete_passenger(passenger_id):
    # 執行 SQL DELETE 語句，根據 passenger_id 把對應的資料從 titanic 資料
    # 表中刪除。
    db.execute(
        "DELETE FROM titanic WHERE PassengerId = ?",
        (passenger_id,)
    )

    # 執行 commit()，把剛剛的 DELETE 操作真正寫入資料庫。
    db.commit()

    # 最後回傳 JSON 格式的資料，包含 message，告訴前端這筆資料已經被刪除
    # 了。
    return jsonify({
        "message": "deleted"
    })

# =====
# 9. 啟動 Flask
# =====

if __name__ == "__main__":
    app.run(
        debug=True,
        host="127.0.0.1",
        port=5000
    )

```

首頁

templates/index.html

```
<!DOCTYPE html>
<html lang="zh-Hant">

<head>
  <meta charset="UTF-8">
  <title>Titanic 乘客資料管理</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
    }

    table {
      border-collapse: collapse;
      width: 100%;
      margin-top: 20px;
    }

    th,
    td {
      border: 1px solid #ccc;
      padding: 8px;
      font-size: 14px;
    }

    th {
      background-color: #f2f2f2;
    }

    .toolbar {
      margin-bottom: 20px;
    }

    .toolbar input,
```

```

        .toolbar button {
            padding: 6px;
            margin-right: 6px;
        }

        a {
            text-decoration: none;
            color: blue;
        }

        .pagination {
            margin-top: 20px;
        }
    </style>
</head>

<body>
    <h1>Titanic 乘客資料管理</h1>

    <div class="toolbar">
        <a href="/passengers/new">新增乘客</a>
        <br><br>

        <input type="text" id="searchInput" placeholder="搜尋姓名">

        <button onclick="loadPassengers(1)">查詢</button>
    </div>

    <div id="summary"></div>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>姓名</th>
                <th>性別</th>
                <th>年齡</th>
                <th>艙等</th>
            </tr>
        </thead>
    </table>

```

```

        <th>是否生還</th>
        <th>票價</th>
        <th>登船港口</th>
        <th>操作</th>
    </tr>
</thead>
<tbody id="passengerTableBody"></tbody>
</table>

<div class="pagination">
    <button onclick="prevPage()">上一頁</button>
    <span id="pageInfo"></span>
    <button onclick="nextPage()">下一頁</button>
</div>

<script>
    // 定義全域變數 currentPage、perPage 和 total，
    // 分別用來記錄目前的頁碼、每頁顯示的資料筆數，以及資料的總筆數。
    let currentPage = 1;
    const perPage = 20;
    let total = 0;

    // 定義一個名為 loadPassengers 的非同步函式，接受一個 page 參數，
    // 預設值為 1。
    async function loadPassengers(page = 1) {
        currentPage = page;

        const search =
document.getElementById("searchInput").value;

        const params = new URLSearchParams({
            page: currentPage,
            per_page: perPage,
            search: search
        });

        const response = await
fetch(`/api/passengers?${params.toString()}`);

```

```

    if (!response.ok) {
        alert("讀取資料失敗");
        return;
    }

    const data = await response.json();
    total = data.total;

    renderTable(data.items);
    renderPagination(data.page, data.per_page, data.total);
}

```

// 定義一個名為 `renderTable` 的函式，接受一個 `items` 參數，這個參數是一個陣列，包含了要顯示在表格中的資料。

```

function renderTable(items) {
    const tbody =
document.getElementById("passengerTableBody");
    tbody.innerHTML = "";

    for (const item of items) {
        const tr = document.createElement("tr");

        tr.innerHTML = `
            <td>${item.PassengerId}</td>
            <td>${escapeHtml(item.Name)}</td>
            <td>${item.Sex}</td>
            <td>${item.Age ?? ""}</td>
            <td>${item.Pclass}</td>
            <td>${item.Survived === 1 ? "生還" : "未生還"}</td>
            <td>${item.Fare}</td>
            <td>${item.Embarked ?? ""}</td>
            <td>
                <a href="/passengers/${item.PassengerId}/edit">
編輯</a>
                |
                <button
onclick="deletePassenger(${item.PassengerId})">刪除</button>
            </td>
        `;
    }
}

```

```

        </td>
      `;

      tbody.appendChild(tr);
    }
  }

  // 定義一個名為 renderPagination 的函式，接受 page、perPage 和
  total 三個參數，分別代表目前的頁碼、每頁顯示的資料筆數，以及資料的總筆數。
  function renderPagination(page, perPage, total) {
    const totalPages = Math.ceil(total / perPage);

    document.getElementById("summary").textContent =
      `共有 ${total} 筆資料`;

    document.getElementById("pageInfo").textContent =
      `第 ${page} 頁 / 共 ${totalPages} 頁`;
  }

  // 定義 prevPage 函式，當使用者點擊「上一頁」按鈕時，會呼叫這個函
  式。

  // 這個函式會檢查目前的頁碼是否大於 1，如果是，就呼叫
  loadPassengers 函式，載入前一頁的資料。
  function prevPage() {
    if (currentPage > 1) {
      loadPassengers(currentPage - 1);
    }
  }

  // 定義 nextPage 函式，當使用者點擊「下一頁」按鈕時，會呼叫這個函
  式。

  // 這個函式會先計算總頁數，然後檢查目前的頁碼是否小於總頁數，
  // 如果是，就呼叫 loadPassengers 函式，載入下一頁的資料。
  function nextPage() {
    const totalPages = Math.ceil(total / perPage);

    if (currentPage < totalPages) {
      loadPassengers(currentPage + 1);
    }
  }

```

```

    }
}

// 定義 deletePassenger 函式，接受一個 passengerId 參數，代表要刪除的乘客 ID。
async function deletePassenger(passengerId) {
    const confirmed = confirm(`確定要刪除
PassengerId=${passengerId} 嗎?`);

    if (!confirmed) {
        return;
    }

    const response = await
fetch(`/api/passengers/${passengerId}`, {
    method: "DELETE"
});

    const result = await response.json();

    if (!response.ok) {
        alert(result.error || "刪除失敗");
        return;
    }

    alert("刪除成功");
    loadPassengers(currentPage);
}

// 定義 escapeHtml 函式，接受一個 text 參數，這個函式的作用是把一些
特殊字元轉換成 HTML 實體，避免 XSS 攻擊。
function escapeHtml(text) {
    if (text === null || text === undefined) {
        return "";
    }

    return String(text)
        .replaceAll("&", "&amp;");
}

```

```

        .replaceAll("<", "&lt;")
        .replaceAll(">", "&gt;")
        .replaceAll("'", "&quot;")
        .replaceAll('"', "&#039;");
    }

    loadPassengers();
</script>
</body>

</html>

```

新增頁面

templates/new.html

```

<!DOCTYPE html>
<html lang="zh-Hant">

<head>
    <meta charset="UTF-8">
    <title>新增乘客</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 30px;
        }

        label {
            display: block;
            margin-top: 10px;
        }

        input,
        select {
            padding: 6px;
            width: 300px;
        }
    </style>

```



```

        button {
            margin-top: 20px;
            padding: 8px 16px;
        }
    </style>
</head>

<body>
    <h1>新增乘客</h1>

    <form id="passengerForm">
        <label>Survived</label>
        <select name="Survived" required>
            <option value="0">0 : 未生還</option>
            <option value="1">1 : 生還</option>
        </select>

        <label>Pclass</label>
        <select name="Pclass" required>
            <option value="1">1 等艙</option>
            <option value="2">2 等艙</option>
            <option value="3">3 等艙</option>
        </select>

        <label>Name</label>
        <input name="Name" required>

        <label>Sex</label>
        <select name="Sex" required>
            <option value="male">male</option>
            <option value="female">female</option>
        </select>

        <label>Age</label>
        <input name="Age" type="number" step="0.01">

        <label>SibSp</label>

```

```

    <input name="SibSp" type="number" min="0" value="0" required>

    <label>Parch</label>
    <input name="Parch" type="number" min="0" value="0" required>

    <label>Ticket</label>
    <input name="Ticket" required>

    <label>Fare</label>
    <input name="Fare" type="number" step="0.0001" min="0"
value="0" required>

    <label>Cabin</label>
    <input name="Cabin">

    <label>Embarked</label>
    <select name="Embarked">
        <option value="">未知</option>
        <option value="C">C</option>
        <option value="Q">Q</option>
        <option value="S">S</option>
    </select>

    <button type="submit">新增</button>
    <a href="/">回首頁</a>
</form>

<script>
    // 監聽 passengerForm 的 submit 事件，當表單被提交時，會觸發這個事件處理器。
    document.getElementById("passengerForm").addEventListener("submit", async function (event) {
        event.preventDefault();

        const form = event.target;
        const formData = new FormData(form);
        const data = Object.fromEntries(formData.entries());

```

```

const response = await fetch("/api/passengers", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
});

const result = await response.json();

if (!response.ok) {
  alert(result.error + "\n" + (result.details ||
[]).join("\n"));
  return;
}

alert("新增成功");
window.location.href = "/";
});
</script>
</body>

</html>

```

編輯頁面

templates/edit.html

```

<!DOCTYPE html>
<html lang="zh-Hant">

<head>
  <meta charset="UTF-8">
  <title>編輯乘客</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
    }
  </style>

```

```

    }

    label {
        display: block;
        margin-top: 10px;
    }

    input,
    select {
        padding: 6px;
        width: 300px;
    }

    button {
        margin-top: 20px;
        padding: 8px 16px;
    }
</style>
</head>

<body>
    <h1>編輯乘客</h1>

    <form id="passengerForm">
        <label>PassengerId</label>
        <input name="PassengerId" disabled>

        <label>Survived</label>
        <select name="Survived" required>
            <option value="0">0 : 未生還</option>
            <option value="1">1 : 生還</option>
        </select>

        <label>Pclass</label>
        <select name="Pclass" required>
            <option value="1">1 等艙</option>
            <option value="2">2 等艙</option>
            <option value="3">3 等艙</option>

```

```

</select>

<label>Name</label>
<input name="Name" required>

<label>Sex</label>
<select name="Sex" required>
  <option value="male">male</option>
  <option value="female">female</option>
</select>

<label>Age</label>
<input name="Age" type="number" step="0.01">

<label>SibSp</label>
<input name="SibSp" type="number" min="0" required>

<label>Parch</label>
<input name="Parch" type="number" min="0" required>

<label>Ticket</label>
<input name="Ticket" required>

<label>Fare</label>
<input name="Fare" type="number" step="0.0001" min="0"
required>

<label>Cabin</label>
<input name="Cabin">

<label>Embarked</label>
<select name="Embarked">
  <option value="">未知</option>
  <option value="C">C</option>
  <option value="Q">Q</option>
  <option value="S">S</option>
</select>

```

```

<button type="submit">更新</button>
<a href="/">回首頁</a>
</form>

<script>
  // 從模板變數中取得 passenger_id，這個變數是在 Flask 的 view
  function 中傳遞給模板的。
  const passengerId = {{ passenger_id }};

  // 定義一個名為 loadPassenger 的非同步函式，這個函式會向後端 API 發
  送 GET 請求，取得指定 passengerId 的乘客資料。
  async function loadPassenger() {
    const response = await
fetch(`/api/passengers/${passengerId}`);

    if (!response.ok) {
      alert("找不到指定乘客");
      window.location.href = "/";
      return;
    }

    const item = await response.json();
    const form = document.getElementById("passengerForm");

    for (const key of Object.keys(item)) {
      if (form.elements[key]) {
        form.elements[key].value = item[key] ?? "";
      }
    }
  }

  // 為 passengerForm 表單綁定 submit 事件的監聽器，當表單被提交時，
  會觸發這個事件處理函式。
  document.getElementById("passengerForm").addEventListener("sub
mit", async function (event) {
    event.preventDefault();

    const form = event.target;

```

```

        const formData = new FormData(form);
        const data = Object.fromEntries(formData.entries());

        const response = await
fetch(`/api/passengers/${passengerId}`, {
    method: "PUT",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify(data)
});

        const result = await response.json();

        if (!response.ok) {
            alert(result.error + "\n" + (result.details ||
[]).join("\n"));
            return;
        }

        alert("更新成功");
        window.location.href = "/";
    });

    loadPassenger();
</script>
</body>

</html>

```